Multi-Source Candidate Data Transformer

Technical Abstract

Problem Statement

Modern recruitment platforms receive candidate information from multiple heterogeneous sources such as recruiter spreadsheets, resumes, ATS exports, and online profiles. These sources often contain duplicate, incomplete, or conflicting information, making it difficult to create a reliable candidate profile. This project implements a configurable data transformation engine that consolidates information from multiple sources into a single canonical candidate profile while preserving provenance, calculating confidence scores, and validating the final output against a predefined schema.

Canonical Candidate Schema

The system transforms all input sources into a unified candidate profile consisting of: Candidate ID, Full Name, Email Addresses, Phone Numbers, Location, Professional Headline, Skills, Work Experience, Education, Provenance Metadata, and Overall Confidence Score. This canonical model enables consistent processing regardless of the source format.

Normalization Strategy

To improve data quality and consistency, the transformation engine performs: phone number normalization to E.164 international format; email standardization (trim, lowercase, duplicate removal); skill canonicalization (e.g., NodeJS, node js, NODE.JS → Node.js); date normalization to YYYY-MM format; country normalization using ISO-3166 Alpha-2 codes; and removal of duplicate values across all supported fields.

Conflict Resolution & Scoring

When conflicting values are encountered, the engine applies configurable source priorities. In the current implementation, Resume PDF > Recruiter CSV. Each extracted field stores its source of origin (provenance), resolution strategy, and confidence score. Fields from higher-priority sources receive higher confidence values. Agreement across multiple sources increases confidence, while conflicting values reduce it.

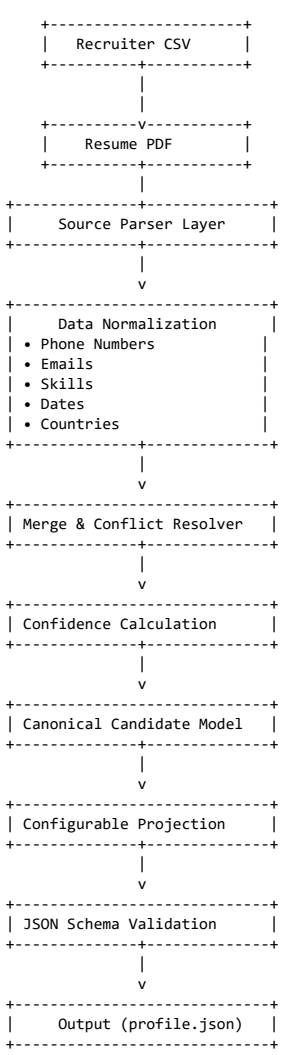
Projection & Validation

The internal canonical model is separated from the output representation using a configurable projection layer. This allows runtime customization of output fields without modifying the transformation logic. The generated JSON is validated against a predefined JSON Schema before being written to disk, ensuring structural consistency and robust error handling.

Conclusion

The proposed solution demonstrates a modular, scalable, and maintainable backend architecture capable of integrating heterogeneous candidate data sources into a unified and validated candidate profile. By leveraging object-oriented design principles, configurable processing, provenance tracking, confidence scoring, and schema validation, the system provides a robust foundation for future extensions.

System Architecture



Technologies Used

Java 17 • Maven • Apache PDFBox • OpenCSV • Jackson • Google libphonenumber • JUnit 5 • SLF4J • Logback